

CSE 151B Milestone Report — Team Name

Ketan Mittal
Department of Data Science
University of California, San Diego
`k1mittal@ucsd.edu`

Anirudh Annabathula
Department of Data Science
University of California, San Diego
`aannabathula@ucsd.edu`

Kaii Bijlani
Department of Data Science
University of California, San Diego
`kbijlani@ucsd.edu`

May 12, 2026

Abstract

This is an in-progress milestone report for our CSE 151B Spring 2026 Math Reasoning Competition entry. The competition gives us 1,126 public math problems (375 multiple-choice and 751 free-form) and a private test set, and asks a language model to answer all of them. So far we have the inference pipeline working end-to-end: we load Qwen3-4B-Thinking-2507 with vLLM and INT8 quantization on a single GPU, generate responses in batches, and score them with the provided `judge.py`. The only method change we have made so far is a new system prompt that tries to match two behaviors of the judge that were not obvious from the starter code: a 10^{-8} relative numerical tolerance, and a boxed-answer extractor that only looks at the text after `</think>` and only keeps the last contiguous group of boxed expressions. On a 20-question test subset this prompt change raised accuracy from 50% (10/20) to 55% (11/20), about a 5 percentage point gain. The clearest example is a temperature-conversion problem where the old prompt produced decimals like 335.93, which fail the strict tolerance, and the new prompt produces exact fractions like $\frac{60467}{180}$, which the judge accepts. We are treating this as preliminary: the bigger lever we are working on next is GRPO fine-tuning of the base model on a Claude-filtered subset of the public dataset, where we use Claude to flag items whose published gold answer looks wrong or ambiguous and train only on the rest. We did submit our current pipeline to the Kaggle leaderboard and scored **0.515** on the private set, which is roughly in line with the 55%

we saw on the dev subset and which we treat as a weak starting baseline rather than a real result.

1 Introduction

Problem Definition. Each test item is either a multiple-choice question (MCQ) with up to ten labeled options, or a free-form question that wants one or more numeric or symbolic values. The model gets the question text (and the option list when there is one), and has to produce a final answer wrapped in a boxed expression. The provided `judger.py` then parses and scores that answer. For free-form items the judger uses a sympy-based comparator with a 10^{-8} relative tolerance; for MCQ items it just expects a single capital letter inside the box. So the task is really a constrained generation problem: the model has to do multi-step math reasoning *and* put the final answer in a specific format the judger can parse.

Problem Significance. Math reasoning is one of the things people most want LLMs to do well — tutoring tools, science assistants, and many engineering workflows assume the model can both compute the right answer and report it without ambiguity. The gap between a model that “gets” a problem internally and one that emits an answer the grading code will accept is a real, practical robustness problem, and that gap is exactly what this competition exposes.

Technical Challenge. This project is harder than it might look for a few reasons. (1) **Data:** the public set covers algebra, calculus, statistics, probability, linear algebra, geometry, number theory, and discrete math, and many of the LaTeX strings are slightly corrupted (e.g., `frac` or `int` without backslashes). (2) **Grader:** the 10^{-8} *relative* tolerance means that a visibly correct decimal like 0.333 is graded wrong against gold $\frac{1}{3}$; the boxed-extraction code only keeps the last contiguous group of boxed expressions in the segment after `</think>`. Either of these can quietly cost points if the prompt does not address them. (3) **Model:** 4B parameters is small for math; the model can lose the thread on harder items, and it likes to restate decimals mid-solution. (4) **Engineering:** we have one consumer-grade GPU, so we use INT8 quantization through BitsAndBytes to fit long thinking traces (up to 32,768 tokens) into the KV cache. (5) **Evaluation:** a full pass over 1,126 questions is slow, so we iterate on small subsets and only do full runs occasionally.

Contributions. The starter notebook `starter_code_cse151b_comp.ipynb` already gives us an end-to-end skeleton: `uv` environment setup, vLLM model loading with INT8 quantization, a minimal MCQ/free-form prompt pair, batched generation, and scoring against the public set. The main thing it does not do is take the judger’s parsing rules seriously — the starter prompts only say “put your answer inside `\boxed{}`” and never mention the 10^{-8} tolerance or the

contiguous-group extraction rule. So far our work just addresses that gap; the rest is still in progress.

- **Contribution A (this milestone):** a grader-aware system prompt for both free-form and MCQ items. It tells the model to keep things in exact symbolic form, to only use a decimal when the exact answer is itself a finite decimal (or the problem asks for one), and to put multi-part answers in a single boxed expression so the judger’s contiguous-group rule is satisfied. On the 20-question test subset this took overall accuracy from 50% to 55% and produced a clear qualitative win on decimal-vs-exact items (Section 4.4).
- **Contribution B (in progress):** scaling the evaluation up to the full 1,126-item public set, running ablations on the prompt’s pieces (exact-symbolic clause, contiguous-box clause, MCQ recompute-on-mismatch), and trying multiple-sample majority voting at the recommended Qwen sampling parameters so we can tell prompt effects apart from sampling noise.
- **Contribution C (planned, next major step):** GRPO [2] fine-tuning of Qwen3-4B-Thinking-2507 on a Claude-filtered subset of the public dataset. We follow the same recipe the Qwen team uses for their own thinking models in the Qwen3 technical report [1], where a cold-start SFT stage is followed by GRPO over a filtered set of query–verifier pairs. The plan is to ask Claude to read each item line by line and flag the ones whose published gold answer looks wrong, ambiguous, or underspecified, and then run GRPO on the items that survive, using judger correctness as the reward. This is meant to attack the kind of errors prompt engineering alone cannot fix (real reasoning mistakes on hard items, not just formatting drift).

2 Related Work (Optional)

LLMs for mathematical reasoning. Open-weight “thinking” models like Qwen3-4B-Thinking-2507 [1] are trained to do their reasoning inside `<think>...</think>` tokens before answering. The Qwen team’s evaluations on MATH-500-style benchmarks recommend specific sampling settings ($\text{temp} = 0.6$, $\text{top-}p = 0.95$, $\text{top-}k = 20$) and a trailing user instruction (“Please reason step by step, and put your final answer within `\boxed{\}`”). We use both of those without changes.

GRPO and RL fine-tuning for math. Group Relative Policy Optimization (GRPO) was introduced in DeepSeekMath [2] as a way to fine-tune reasoning models with a verifier signal, and it showed that using a deterministic grader as the reward function can noticeably improve a base model’s math accuracy. The same recipe was then used in Qwen2.5-Math [3] and again in the Qwen3 technical report [1], where the team collected roughly 4,000 query–verifier pairs and used

GRPO to update the model after a small cold-start SFT phase. Because we already have a working judger that returns 0/1 per item and the same backbone Qwen3 model the technical report describes, that recipe carries over to our setup almost directly — which is why we are planning it as our main next step.

Prompt engineering for structured output. A consistent finding in recent LLM-with-grader pipelines is that spelling out the grader’s parsing rules in the system prompt helps much more than describing the answer’s semantics alone. Our Contribution A is an instance of this idea, specialized to the 10^{-8} relative tolerance and the contiguous-boxed-group rule of the provided judger.

3 Methods

Problem Setting. We treat each item as a tuple (q, O, y) where q is the question text, O is the option list (empty for free-form items), and y is the gold answer (a capital letter for MCQ; a list of one or more numeric/symbolic strings for free-form). The model defines a conditional distribution $p_\theta(r \mid s(q, O), u(q, O))$ over response strings r , where s and u are the system and user messages built by `build_prompt`. The judger $\mathcal{J}(r, y, O) \in \{0, 1\}$ is the function in `judger.py`: it pulls the last contiguous group of boxed expressions from r (only looking at the text after `</think>`), normalizes the LaTeX, and compares to y using `sympy` with a 10^{-8} relative tolerance on numerical values. We want to maximize the expected accuracy $\mathbb{E}_{(q, O, y) \sim \mathcal{D}} \mathbb{E}_{r \sim p_\theta} [\mathcal{J}(r, y, O)]$. At this milestone we do *not* change θ (no fine-tuning yet); we only change s . Fine-tuning is planned for the next phase (see Contribution C).

The dataset is `data/public.jsonl` (1,126 items: 375 MCQ, 751 free-form). For fast iteration we use the first 20 items as a dev subset. The model sees the raw question and option strings, and we keep its full response as-is. We do not pre- or post-process the strings beyond what the judger already does at scoring time.

Idea Summary. The starter prompt is short: "solve step by step" and "put your final answer inside `\boxed{\}`." We noticed it does not mention two things the judger actually cares about: (i) **decimal drift**, where the model gets $\frac{565}{9}$ internally but writes " ≈ 62.78 " inside the box and fails the 10^{-8} tolerance against the exact fraction; and (ii) **boxed fragmentation**, where the model writes a final non-contiguous box after restating an earlier answer and the judger silently drops everything except the last group. Our new prompt addresses both. For free-form items we explicitly tell the model to prefer exact symbolic forms (e.g., `\frac{1}{3}` over `0.333`, `\sqrt{2}` over `1.414`), list the carve-outs (decimals are fine when the exact answer is already a finite decimal, or when the problem explicitly asks for one), and ban `\approx` inside the box. For multi-part answers we require a single boxed expression with comma-separated values in the order asked, which is the easy way to never trip the contiguous-group rule. For MCQ items we tell

the model to compute the exact symbolic answer first and only convert to a decimal at the end to match against the options, and to recompute from scratch (rather than picking the closest-looking option) when nothing matches.

There is a real risk that this prompt is too prescriptive and ends up distracting the model from the actual math. We try to mitigate that by keeping the reasoning instructions short and putting all the hard format rules in their own "Final-answer format" section so the model can read past them once it knows the rules. If the prompt does not generalize past the dev subset we have a fallback: take the model's response and re-prompt it to re-cast just the final box in exact symbolic form, which keeps the original reasoning intact and only patches the formatting.

After we have run the prompt-engineering ablations in earnest, the next planned step is GRPO fine-tuning. The setup we have in mind is to first run Claude over the public dataset to filter out items whose gold answer is wrong, ambiguous, or underspecified, then run GRPO on the surviving items with the judge as the reward function (+1 for correct, 0 otherwise). The Claude-based filter is important because training a small model with RL against noisy gold answers tends to make things worse, not better, so we want to make sure the supervision signal is clean before we update any weights.

4 Experiments

4.1 Baselines

We compare two prompts. Everything else — model, quantization, sampling parameters, dataset subset — is the same for both runs:

- **Starter prompt** (baseline, from `starter_code_cse151b.comp.ipynb`): a two-sentence system message that asks the model to solve step by step and put the final answer "inside `\boxed{}`" with comma-separated values for multi-answer items. It says nothing about decimal vs. exact form or about the contiguous-group rule, and the MCQ version just says "output ONLY the letter."
- **Refined prompt (ours)**: the `SYSTEM_PROMPT_MATH` and `SYSTEM_PROMPT_MCQ` we describe in Section 3, plus the Qwen team's recommended user-side suffix "Please reason step by step, and put your final answer within `\boxed{}`."

We do not include simpler models like a multi-layer perceptron or linear regression as baselines, because they cannot really do free-form math reasoning over natural-language problem statements in any meaningful way. Comparing prompt variants of the same backbone model is what made sense to us at this stage.

4.2 Evaluation

We score with the provided judge: `Judge.auto_judge` for free-form items, and an exact-letter match (after extracting the boxed letter) for MCQ items. This

is the same scoring the starter notebook uses. For this milestone we report accuracy on the first 20 items of `public.jsonl`, split by question type. We are saving the full public set and any Kaggle submissions for the final report. We also do qualitative case studies on items whose correctness flipped between the two prompts, since those are the most informative about what the prompt change is and is not buying us.

4.3 Implementation details

- **Hardware.** A single CUDA GPU (`CUDA_VISIBLE_DEVICES=0`). Generating responses for the 20-question subset takes about 2–3 minutes after the model is loaded; we estimate a full-public-set run (1,126 questions) at roughly 90–120 minutes depending on how long the responses end up being. We are not training yet, so there is no per-epoch time to report.
- **Model and quantization.** Qwen/Qwen3-4B-Thinking-2507 loaded through vLLM with `quantization="bitsandbytes"` and `load_format="bitsandbytes"` (INT8 weights), `gpu_memory_utilization=0.85`, `max_model_len=8192`, `max_num_seqs=256`, `max_num_batched_tokens=32768`, `enable_prefix_caching=False`, `trust_remote_code=True`.
- **Sampling.** We have not deliberately tuned sampling at this milestone. Our active inference cell sets `temperature=0.6`, `top_p=0.95`, `top_k=20`, `repetition_penalty=1.0`, and `max_tokens=32,768`, and otherwise leaves things at their defaults. These values happen to line up with what the Qwen team recommends for their thinking models, but we have not yet run a controlled comparison against alternatives — intentionally validating the recommended settings (and varying them to see what actually matters) is on our list for the next phase.
- **Keeping the comparison fair.** Both prompt variants use the same sampling parameters, the same chat template (the tokenizer’s default `apply_chat_template` with `add_generation_prompt=True`), and the same scoring code. The only thing that changes between the two runs is the contents of `SYSTEM_PROMPT_MATH` and `SYSTEM_PROMPT_MCQ`.
- **Caveat.** We do not fix a random seed for vLLM at this milestone, so per-question outcomes have some sampling noise. Section 4.4 talks about how this shows up.

4.4 Results

Result 1: the new prompt raises accuracy by +5 percentage points on the dev subset. Table 1 shows the head-to-head on the 20-question test subset. Overall accuracy goes from 50% (10/20) to 55% (11/20). The MCQ split is unchanged at 4/9; the free-form split improves from 6/11 to 7/11.

Table 1: Accuracy on the 20-question test subset (first 20 items of `public.jsonl`). Both rows use Qwen3-4B-Thinking-2507 (INT8 via vLLM) with the same sampling parameters; only the system prompt is different.

Prompt	MCQ ($n = 9$)	Free-form ($n = 11$)	Overall ($n = 20$)
Starter	4 / 9	6 / 11	10 / 20 (50.0%)
Refined (ours)	4 / 9	7 / 11	11 / 20 (55.0%)

Result 2: a clear qualitative example of decimal-vs-exact. The clearest example of what the new prompt is doing is item `id=5`, a free-form temperature-conversion problem with `gold = [62.77777777777778, 335.9277777777778, 604.67]`. With the starter prompt the model writes `[62.78, 335.93, 604.67]`; the middle value differs from gold by about 10^{-4} relative, which is way outside the 10^{-8} tolerance, so the whole item is marked wrong. With the refined prompt the model writes `[ $\frac{565}{9}$ ,  $\frac{60467}{180}$ ,  $\frac{60467}{100}$ ]`; each fraction equals the gold decimal exactly under sympy, so the item is marked correct. This is exactly the failure mode the new prompt was designed to fix, and seeing it work on a real item is the main reason we believe the prompt change is helping.

Result 3 (negative finding): the gain is small enough that sampling noise still matters. Three of the 20 items flipped between the two runs: `id=5` (free-form, $F \rightarrow T$) and `id=19` (MCQ, $F \rightarrow T$) are wins, but `id=13` (MCQ, $T \rightarrow F$) is a regression. Item 13 is a triple-integral MCQ; the refined-prompt run loses the thread mid-reasoning rather than producing a malformed final answer, so we do not think the prompt is actually making the model worse there — it is more likely just sampling variance, since temperature is 0.6 and we did not seed. The honest read of the +1 net gain on $n = 20$ is that it is a noisy point estimate, and we plan to tighten it in the final report by (a) running on the full public set and (b) averaging over multiple sampling seeds.

5 Discussion

At this point in the project we have an inference pipeline that runs end-to-end (model loading, batched generation, judger-based scoring) and one method change — the new system prompt — that removes a specific kind of avoidable loss (decimal drift on free-form items). Most of the work is still ahead of us, and the rest of this section is about what we still need to do to make it better.

Achievements. We got the starter pipeline running locally on our own hardware, read through `judger.py` carefully enough to understand the two behaviors that surprised us (the 10^{-8} relative tolerance and the post-`</think>` contiguous-boxed-group extraction), and used that understanding to design a new system prompt that explicitly respects both. On the 20-question test subset the refined

prompt gave a +5 percentage point gain (50% $\rightarrow$ 55%) under matched sampling parameters, with the win coming from the free-form split (+1 of 11). We also submitted the refined pipeline’s private-set predictions (943 items) to Kaggle and scored **0.515** on the private leaderboard. This is honestly not a strong result: it lines up with the 55% we saw on the dev subset and confirms that prompt engineering alone is not enough to get us where we want to be. We are reporting it as our current baseline rather than as a finished result — the rest of this section is about why the score is where it is and what we are doing next to improve it.

Bottlenecks and Mitigation. The biggest issue right now is statistical: 20 questions is just not enough to tell prompt effects apart from sampling noise (see Result 3). The plan is to run on the full 1,126-item public set and average over multiple sampling seeds for the final report. The second issue is wall-clock time — a full-public-set pass takes roughly 90–120 minutes on our hardware. We are planning to do a single full sweep and then iterate on cheaper post-hoc rewrites rather than re-running inference each time. The third is the LaTeX corruption in some of the public-set questions (`frac` without a backslash, etc.); the judger’s `normalize_answer` handles most of it, but if we see a recurring error class we will add a small pre-processor to the question text.

Plans Forward. Between now and the final report we plan to:

- Run a full 1,126-item public-set evaluation under both prompts so the comparison is on real numbers, not 20.
- Ablate the new prompt’s individual pieces (exact-symbolic clause, contiguous-box clause, MCQ recompute-on-mismatch clause) to figure out which parts are actually doing the work.
- Deliberately validate the Qwen team’s recommended thinking-mode sampling settings. We currently have those values in our code but have not run a controlled comparison; we plan to sweep `temperature`, `top_p`, `top_k`, and `min_p` on a fixed subset and check whether the recommended values actually help in our pipeline.
- Try multi-sample majority voting at $k \in \{3, 5\}$, applying the judger’s boxed extractor to each sample independently. This should reduce the sampling noise we saw in Result 3.
- Build a small post-hoc rewriter that takes the model’s response and re-emits just the final box in clean exact symbolic form. That way reasoning errors and formatting errors stop being coupled.
- **(Main next step) Run GRPO fine-tuning on a Claude-filtered subset of the public dataset.** We use Claude as a verifier: for each public-set item, Claude reads the question, the published gold answer(s),

and the listed options, and tells us whether the gold answer is plausibly correct, ambiguous, or flat-out wrong. We drop everything Claude flags as wrong or ambiguous and keep only the high-confidence items. Then we run GRPO on Qwen3-4B-Thinking-2507 with that filtered set, using the judge’s 0/1 correctness signal as the reward. The hope is that this catches the errors prompt engineering cannot fix — real reasoning mistakes on hard problems, not just bad formatting.

This is broadly consistent with the original proposal (prompt engineering as the first step, fine-tuning as the next). The two deviations from the proposal are (a) the multi-sample voting plan, which we added once we saw the sampling noise in Result 3, and (b) the Claude-based dataset filter, which we added once we noticed some of the public-set gold answers look suspect.

Team Responsibilities.

- **Ketan Mittal** — grader analysis and prompt design; ran the starter-vs-refined comparison; produced the private-set prediction file.
- **Anirudh Annabathula** — inference infrastructure (vLLM, INT8 quantization, batched generation); will own the GRPO training setup.
- **Kaii Bijlani** — dataset analysis and qualitative case-study pipeline; will own the Claude-based dataset filter.

Timelines.

- **Week 2:** Form team; reproduce the starter notebook locally.
- **Week 3:** Read and document `judge.py`; identify the 10^{-8} relative-tolerance and contiguous-boxed-group behaviors.
- **Week 4:** Draft and check the grader-aware system prompt on the 20-question test subset; produce the results in Table 1; generate the first private-set prediction file.
- **Week 5 (current):** Submit milestone report.
- **Week 6:** Run the full 1,126-item public-set evaluation under both prompts; record per-question correctness so we can ablate later.
- **Week 7:** Prompt-component ablations; multi-sample majority-voting experiments.
- **Week 8:** Run Claude as a verifier over the public set and build the filtered training subset; stand up the GRPO training loop.
- **Week 9:** GRPO fine-tuning runs; lock final results; resubmit to Kaggle and try to beat the current 0.515 baseline.
- **Week 10:** Write up and submit the final report.

LLM Usage

We used an LLM assistant (Anthropic Claude) for two things: (i) help with wording, mostly tightening up phrasing in this report and in our system prompts, and (ii) help making sense of hard-to-read error messages in LaTeX and Python while we were writing the report and building the pipeline. All the technical decisions, code, experimental runs, and numbers in this report are ours, and we take full responsibility for the contents.

References

- [1] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. URL <https://arxiv.org/abs/2505.09388>.
- [2] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024. URL <https://arxiv.org/abs/2402.03300>.
- [3] An Yang et al. Qwen2.5-Math technical report: Toward mathematical expert model via self-improvement. *arXiv preprint arXiv:2409.12122*, 2024. URL <https://arxiv.org/abs/2409.12122>.